

Fengping River Hydropower — Test

Assignment 6 — 3 simple function tests

[MAYA SCANLON , WILLIAM LIEN]

2026-06-14

Table of contents

1 Function 1: fill_daily_na() — daily flow data	1
2 Function 2: compute_npv() — NPV / CAPEX	2
3 Function 3: fit_rating_curve() — sediment data format & fit	2

This document is a newly organised document on the test cases for the three functions we have created in the module. The tests checks the input and output format. The tests are written using the `testthat` package, which provides a framework for writing unit tests in R. Each function is tested for its expected behavior, including handling of edge cases and invalid inputs.

```
library(tidyverse)
library(here)
library(testthat)

source(here("module", "hydro_reservoir.R"))
source(here("module", "hydro_sediment.R"))
source(here("module", "hydro_finance.R"))
```

1 Function 1: fill_daily_na() — daily flow data

```
test_that("fill_daily_na: detects and fills NA gaps, output is reasonable", {

  # Synthetic flow data with a gap (NA) in the middle
  synth <- data.frame(
    date = seq(as.Date("2020-01-01"), as.Date("2020-01-05"), by = "day"),
    Q_cms = c(10, 12, NA, 16, 18)
  )

  # Confirm the test data actually has a gap
```

```

expect_true(any(is.na(synth$Q_cms)))

result <- fill_daily_na(synth, method = "linear")

# Gap is gone after filling
expect_true(all(!is.na(result$Q_cms)))

# Filled flow values are physically reasonable (non-negative)
expect_true(all(result$Q_cms >= 0))

# Overall distribution of filled values
summary(result$Q_cms)
})

```

Test passed with 3 successes.

2 Function 2: compute_npv() — NPV / CAPEX

```

test_that("compute_npv: basic correctness and input checks", {

  # At r = 0, NPV is just the sum of cash flows (hand-checkable)
  cf <- c(-1000, 300, 300, 300, 300)
  expect_equal(compute_npv(cf, discount_rate = 0), sum(cf))

  # A single positive value at year 0 needs no discounting
  expect_equal(compute_npv(c(100, 0), discount_rate = 0.1), 100)

  # Higher discount rate -> lower NPV (for positive future flows)
  npv_low <- compute_npv(cf, discount_rate = 0.01)
  npv_high <- compute_npv(cf, discount_rate = 0.15)
  expect_true(npv_low > npv_high)

  # Invalid input (length < 2) should error, per stopifnot()
  expect_error(compute_npv(c(-1000), discount_rate = 0.05))
})

```

Test passed with 4 successes.

3 Function 3: fit_rating_curve() — sediment data format & fit

```

test_that("fit_rating_curve: input format and fitted curve are valid", {

  # Sediment sampling data: required columns Discharge_CMS, Sus_Load_MTDy, flag
  sed_sample <- data.frame(
    Discharge_CMS = c(10, 20, 30, 50, 80, 120),
    Sus_Load_MTDy = c(50, 150, 300, 700, 1500, 3000),
    flag          = "ok"
  )

  # Required columns are present
  expect_true(all(c("Discharge_CMS", "Sus_Load_MTDy", "flag") %in% names(sed_sample)))

  # At least 5 valid rows for the regression
  valid_rows <- sed_sample$Discharge_CMS > 0 & sed_sample$Sus_Load_MTDy > 0
  expect_true(sum(valid_rows) >= 5)

  # Fitted curve returns a list with 'a' and 'b'
  rc <- fit_rating_curve(sed_sample, plot_fit = FALSE)
  expect_true(all(c("a", "b") %in% names(rc)))

  # 'b' (power-law exponent) is positive: sediment load increases with flow
  expect_true(rc$b > 0)
})

```

Test passed with 4 successes.